JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Enum Class Month

java.lang.Object
    java.lang.Enum<Month>
        java.time.Month

All Implemented Interfaces:

Serializable, Comparable<Month>, Constable, TemporalAccessor, TemporalAdjuster

---

```
public enum Month
extends Enum<Month>
implements TemporalAccessor, TemporalAdjuster
```

A month-of-year, such as 'July'.

`Month` is an enum representing the 12 months of the year - January, February, March, April, May, June, July, August, September, October, November and December.

In addition to the textual enum name, each month-of-year has an `int` value. The `int` value follows normal usage and the ISO-8601 standard, from 1 (January) to 12 (December). It is recommended that applications use the enum rather than the `int` value to ensure code clarity.

Do not use `ordinal()` to obtain the numeric representation of `Month`. Use `getValue()` instead.

This enum represents a common concept that is found in many calendar systems. As such, this enum may be used by any calendar system that has the month-of-year concept defined exactly equivalent to the ISO-8601 calendar system.

Implementation Requirements:

This is an immutable and thread-safe enum.

Since:

1.8

Nested Class Summary

Nested classes/interfaces declared in class Enum
Enum.EnumDesc<E>

Enum Constant Summary

Enum Constants

Enum Constant	Description
APRIL	The singleton instance for the month of April with 30 days.
AUGUST	The singleton instance for the month of August with 31 days.
DECEMBER	The singleton instance for the month of December with 31 days.
FEBRUARY	The singleton instance for the month of February with 28 days, or 29 in a leap year.
JANUARY	The singleton instance for the month of January with 31 days.
JULY	The singleton instance for the month of July with 31 days.
JUNE	The singleton instance for the month of June with 30 days.
MARCH	The singleton instance for the month of March with 31 days.
MAY	The singleton instance for the month of May with 31 days.
NOVEMBER	The singleton instance for the month of November with 30 days.
OCTOBER	The singleton instance for the month of October with 31 days.
SEPTEMBER	The singleton instance for the month of September with 30 days.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description

Temporal	**adjustInto**(Temporal temporal)	Adjusts the specified temporal object to have this month-of-year.
int	**firstDayOfYear**(boolean leapYear)	Gets the day-of-year corresponding to the first day of this month.
Month	**firstMonthOfQuarter**()	Gets the month corresponding to the first month of this quarter.
static Month	**from**(TemporalAccessor tempor	Obtains an instance of Month from a temporal object.
int	**get**(TemporalField field)	Gets the value of the specified field from this month-of-year as an int.
String	**getDisplayName**(TextStyle style, Locale locale)	Gets the textual representation, such as 'Jan' or 'December'.
long	**getLong**(TemporalField field)	Gets the value of the specified field from this month-of-year as a long.
int	**getValue**()	Gets the month-of-year int value.
boolean	**isSupported**(TemporalField field)	Checks if the specified field is supported.
int	**length**(boolean leapYear)	Gets the length of this month in days.
int	**maxLength**()	Gets the maximum length of this month in days.
int	**minLength**()	Gets the minimum length of this month in days.
Month	**minus**(long months)	

		Returns the month-of-year that is the specified number of months before this one.
static Month	**of**(int month)	Obtains an instance of Month from an int value.
Month	**plus**(long months)	Returns the month-of-year that is the specified number of months after this one.
<R> R	**query**(TemporalQuery<R> query)	Queries this month-of-year using the specified query.
ValueRange	**range**(TemporalField field)	Gets the range of valid values for the specified field.
static Month	**valueOf**(String name)	Returns the enum constant of this class with the specified name.
static Month[]	**values**()	Returns an array containing the constants of this enum class, in the order they are declared.

Methods declared in class Enum

clone, compareTo, describeConstable, equals, finalize, getDeclaringClass, hashCode, name, ordinal, toString, valueOf

Methods declared in class Object

getClass, notify, notifyAll, wait, wait, wait

Enum Constant Details

JANUARY

public static final Month JANUARY

The singleton instance for the month of January with 31 days. This has the numeric value of 1.

FEBRUARY

```
public static final Month FEBRUARY
```

The singleton instance for the month of February with 28 days, or 29 in a leap year. This has the numeric value of 2.

MARCH

```
public static final Month MARCH
```

The singleton instance for the month of March with 31 days. This has the numeric value of 3.

APRIL

```
public static final Month APRIL
```

The singleton instance for the month of April with 30 days. This has the numeric value of 4.

MAY

```
public static final Month MAY
```

The singleton instance for the month of May with 31 days. This has the numeric value of 5.

JUNE

```
public static final Month JUNE
```

The singleton instance for the month of June with 30 days. This has the numeric value of 6.

JULY

```
public static final Month JULY
```

The singleton instance for the month of July with 31 days. This has the numeric value of 7.

AUGUST

```
public static final Month AUGUST
```

The singleton instance for the month of August with 31 days. This has the numeric value of 8.

SEPTEMBER

```
public static final Month SEPTEMBER
```

The singleton instance for the month of September with 30 days. This has the numeric value of 9.

OCTOBER

```
public static final Month OCTOBER
```

The singleton instance for the month of October with 31 days. This has the numeric value of 10.

NOVEMBER

```
public static final Month NOVEMBER
```

The singleton instance for the month of November with 30 days. This has the numeric value of 11.

DECEMBER

```
public static final Month DECEMBER
```

The singleton instance for the month of December with 31 days. This has the numeric value of 12.

Method Details

values

```
public static Month[] values()
```

Returns an array containing the constants of this enum class, in the order they are declared.

Returns:

an array containing the constants of this enum class, in the order they are declared

valueOf

```
public static Month valueOf(String name)
```

Returns the enum constant of this class with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this class. (Extraneous whitespace characters are not permitted.)

Parameters:

`name` - the name of the enum constant to be returned.

Returns:

the enum constant with the specified name

Throws:

`IllegalArgumentException` - if this enum class has no constant with the specified name

`NullPointerException` - if the argument is null

of

```
public static Month of(int month)
```

Obtains an instance of `Month` from an `int` value.

`Month` is an enum representing the 12 months of the year. This factory allows the enum to be obtained from the `int` value. The `int` value follows the ISO-8601 standard, from 1 (January) to 12 (December).

Parameters:

`month` - the month-of-year to represent, from 1 (January) to 12 (December)

Returns:

the month-of-year, not null

Throws:

`DateTimeException` - if the month-of-year is invalid

from

```
public static Month from(TemporalAccessor temporal)
```

Obtains an instance of `Month` from a temporal object.

This obtains a month based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `Month`.

The conversion extracts the `MONTH_OF_YEAR` field. The extraction is only permitted if the temporal object has an ISO chronology, or can be converted to a `LocalDate`.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `Month::from`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the month-of-year, not null

Throws:

`DateTimeException` - if unable to convert to a `Month`

getValue

```
public int getValue()
```

Gets the month-of-year `int` value.

The values are numbered following the ISO-8601 standard, from 1 (January) to 12 (December).

Returns:

the month-of-year, from 1 (January) to 12 (December)

getDisplayName

```
public String getDisplayName(TextStyle style,
                             Locale locale)
```

Gets the textual representation, such as 'Jan' or 'December'.

This returns the textual name used to identify the month-of-year, suitable for presentation to the user. The parameters control the style of the returned text and the locale.

If no textual mapping is found then the `numeric value` is returned.

Parameters:

`style` - the length of the text required, not null

`locale` - the locale to use, not null

Returns:

the text value of the month-of-year, not null

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this month-of-year can be queried for the specified field. If false, then calling the `range` and `get` methods will throw an exception.

If the field is `MONTH_OF_YEAR` then this method returns true. All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field` - the field to check, null returns false

Returns:

true if the field is supported on this month-of-year, false if not

range

```
public ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This month is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is `MONTH_OF_YEAR` then the range of the month-of-year, from 1 to 12, will be returned. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

UnsupportedTemporalTypeException - if the field is not supported

get

```
public int get(TemporalField field)
```

Gets the value of the specified field from this month-of-year as an `int`.

This queries this month for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is `MONTH_OF_YEAR` then the value of the month-of-year, from 1 to 12, will be returned. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field, within the valid range of values

Throws:

`DateTimeException` - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

`UnsupportedTemporalTypeException` - if the field is not supported or the range of values exceeds an `int`

`ArithmeticException` - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this month-of-year as a `long`.

This queries this month for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is MONTH_OF_YEAR then the value of the month-of-year, from 1 to 12, will be returned. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

getLong in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained

UnsupportedTemporalTypeException - if the field is not supported

ArithmeticException - if numeric overflow occurs

plus

```
public Month plus(long months)
```

Returns the month-of-year that is the specified number of months after this one.

The calculation rolls around the end of the year from December to January. The specified period may be negative.

This instance is immutable and unaffected by this method call.

Parameters:

months - the months to add, positive or negative

Returns:

the resulting month, not null

minus

```
public Month minus(long months)
```

Returns the month-of-year that is the specified number of months before this one.

The calculation rolls around the start of the year from January to December. The specified period may be negative.

This instance is immutable and unaffected by this method call.

Parameters:

`months` - the months to subtract, positive or negative

Returns:

the resulting month, not null

length

```
public int length(boolean leapYear)
```

Gets the length of this month in days.

This takes a flag to determine whether to return the length for a leap year or not.

February has 28 days in a standard year and 29 days in a leap year. April, June, September and November have 30 days. All other months have 31 days.

Parameters:

`leapYear` - true if the length is required for a leap year

Returns:

the length of this month in days, from 28 to 31

minLength

```
public int minLength()
```

Gets the minimum length of this month in days.

February has a minimum length of 28 days. April, June, September and November have 30 days. All other months have 31 days.

Returns:

the minimum length of this month in days, from 28 to 31

maxLength

```
public int maxLength()
```

Gets the maximum length of this month in days.

February has a maximum length of 29 days. April, June, September and November have 30 days. All other months have 31 days.

Returns:

the maximum length of this month in days, from 29 to 31

firstDayOfYear

`public int firstDayOfYear(boolean leapYear)`

Gets the day-of-year corresponding to the first day of this month.

This returns the day-of-year that this month begins on, using the leap year flag to determine the length of February.

Parameters:

`leapYear` - true if the length is required for a leap year

Returns:

the day of year corresponding to the first day of this month, from 1 to 336

firstMonthOfQuarter

`public Month firstMonthOfQuarter()`

Gets the month corresponding to the first month of this quarter.

The year can be divided into four quarters. This method returns the first month of the quarter for the base month. January, February and March return January. April, May and June return April. July, August and September return July. October, November and December return October.

Returns:

the first month of the quarter corresponding to this month, not null

query

`public <R> R query(TemporalQuery<R> query)`

Queries this month-of-year using the specified query.

This queries this month-of-year using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

R - the type of the result

Parameters:

query - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

DateTimeException - if unable to query (defined by the query)

ArithmeticException - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have this month-of-year.

This returns a temporal object of the same observable type as the input with the month-of-year changed to be the same as this.

The adjustment is equivalent to using Temporal.with(TemporalField, long) passing ChronoField.MONTH_OF_YEAR as the field. If the specified temporal object does not use the ISO calendar system then a DateTimeException is thrown.

In most cases, it is clearer to reverse the calling pattern by using Temporal.with(TemporalAdjuster):

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisMonth.adjustInto(temporal);
temporal = temporal.with(thisMonth);
```

For example, given a date in May, the following are output:

```
dateInMay.with(JANUARY);    // four months earlier
dateInMay.with(APRIL);      // one months earlier
dateInMay.with(MAY);        // same date
dateInMay.with(JUNE);       // one month later
dateInMay.with(DECEMBER);   // seven months later
```

This instance is immutable and unaffected by this method call.

Specified by:

adjustInto in interface TemporalAdjuster

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.